

Graph HyperNetworks for Language Model Initialization

Anshul Kiyawat

Rochester Institute of Technology

Rochester, USA

ak3748@rit.edu

Abstract

We investigate whether Graph HyperNetworks (GHNs) can generate effective initialization weights for Transformer-based language models compared to standard methods. GHNs are meta-models that predict neural network parameters by processing computational graph representations. While successful in computer vision, their application to language model initialization remains unexplored. We train a Graphormer-based GHN on diverse small Transformer architectures and evaluate on nine held-out models on WikiText-2. Our experiments compare two GHN-based initialization schemes—tiling (GHN-T) and interpolation (GHN-I)—against a strong variance-preserving Transformer baseline (Default), measuring both convergence speed and perplexity. We find that GHN-based initializations produce functional weights that enable stable training across all architectures, but they consistently underperform Default: GHN-I is typically 1-2 epochs slower to converge and yields 10-30% worse test perplexity, while GHN-T lags further behind. Nevertheless, GHN-I significantly improves over random initialization in zero-shot and early-epoch settings. This work represents the first investigation of GHN-based initialization for language models and provides insights into the challenges and opportunities of meta-learning initialization for Transformers.

1 Introduction

The initialization of neural network parameters has long been recognized as a critical factor in determining training dynamics, convergence speed, and final model performance [4, 5]. Traditional initialization methods such as Xavier/Glorot [4] and He initialization [5] provide heuristic-based approaches that rely on preserving variance across layers during forward and backward passes. While these methods have proven effective across various architectures, they remain fundamentally agnostic to the specific computational structure of the target network and treat all architectures within a class uniformly.

Transformer-based language models [10], which have become the dominant paradigm for natural language processing tasks, present unique architectural characteristics including self-attention mechanisms, multi-head attention patterns, and complex residual connections. Training these models from scratch requires substantial computational resources—pretraining large language models like GPT-2 [9] and BERT [3] can demand hundreds of GPU-hours even for modest configurations. The question naturally arises: can we leverage architectural structure to generate superior initialization weights that accelerate training and improve convergence properties?

Recent work has demonstrated the potential of meta-learning approaches to amortize neural network training costs. Graph HyperNetworks (GHNs) [6, 7, 13] represent a promising paradigm where a meta-model learns to predict initialization parameters for

target networks by processing their computational graph representations. The key insight is that the topology and structural properties of a neural network—encoded as a graph with node features representing layer types, dimensions, and activations—contain rich information that can inform better parameter initialization.

While GHNs have shown remarkable success in computer vision tasks, with GHN-3 [7] achieving strong performance on ImageNet (reporting significant one-epoch gains over random initialization on the PYTORCH-10 split) and successfully scaling to predict parameters for diverse ImageNet models, their application to language model initialization remains unexplored to our knowledge. GHN-3 replaced GHN-2’s GatedGNN with Graphormer-style Transformer layers [11], introduced separate forward/backward edge embeddings, and added explicit regularization on predicted parameters during training. The architectural patterns in Transformers used for language modeling—particularly the interplay between attention mechanisms, feed-forward networks, and residual connections—differ substantially from convolutional architectures, motivating investigation into whether GHNs can effectively transfer to the language modeling domain.

Moreover, accelerating language model training has immediate practical implications. Training acceleration techniques such as progressive layer stacking [15], which achieves over 110% speedup for BERT, and large batch optimization [12], which reduces BERT pretraining from three days to 76 minutes, have demonstrated significant efficiency gains. These approaches are orthogonal to improved initialization and could potentially be combined with GHN-based methods. If GHN-based initialization can similarly reduce convergence time while maintaining or improving final performance, it could democratize access to language model development for resource-constrained researchers and practitioners.

We investigate whether Graph HyperNetworks can generate effective initialization weights for Transformer-based language models. We encode Transformer architectures as directed computational graphs where nodes represent operations with features capturing layer type, dimensionality, and activation functions, while edges encode data flow and residual connections. We train a graph neural network to predict initialization weights for diverse small Transformer variants (2-6 layers, 128-512 hidden dimensions) and evaluate on held-out architectures using WikiText-2 [8]. We compare GHN-T and GHN-I against a strong variance-preserving Transformer baseline (Default), using test perplexity and convergence speed as primary metrics, and additionally compare against random initialization in a limited zero-shot and early-epoch setting.

Our main contributions are: (1) the first systematic evaluation of Graph HyperNetworks for initializing Transformer language models, (2) a graph representation and GHN training setup tailored to Transformer architectures, and (3) an empirical analysis showing that GHN-based initialization produces functional weights but does

not yet match strong Transformer baselines, highlighting concrete failure modes and future directions.

The remainder of this paper is organized as follows: Section 2 reviews related work on parameter prediction, language model initialization, and training acceleration; Section 3 details our graph representation, GHN architecture, and evaluation protocol; Section 4 presents comprehensive results comparing GHN initialization to baselines; Section 5 analyzes factors affecting GHN performance; and Section 7 concludes with limitations and future directions.

2 Related Work

Graph HyperNetworks and Parameter Prediction. Zhang et al. [13] introduced GHNs for neural architecture search, demonstrating that graph neural networks could predict parameters for diverse architectures. Knyazev et al. [6] extended this to parameter prediction for unseen deep architectures, showing that a single GHN trained on small CNNs could generalize to larger networks, achieving 77% top-1 accuracy on CIFAR-10 without additional training. Most recently, GHN-3 [7] scaled this approach using Graphormer-based architectures with separate forward/backward edge embeddings and group ℓ_1 - ℓ_2 regularization, successfully predicting parameters for diverse ImageNet models and achieving faster convergence when used for initialization.

Language Model Initialization and Optimization. Beyond classical schemes like Xavier/Glorot [4] and He initialization [5], recent work has explored improved initialization strategies. FixUp [14] eliminates the need for normalization through careful initialization scaling. MetaInit [2] and GradInit [16] optimize initialization to reduce gradient variance, while NIO [?] learns network initializations from data. These methods are complementary to GHN-based approaches, which additionally leverage architectural structure.

Training Acceleration for Transformers. Complementary work has explored various approaches to accelerate Transformer training. Progressive Stacking 2.0 [15] grows model depth during training for faster convergence. The LAMB optimizer [12] enables massive batch sizes without accuracy loss. EarlyBERT [1] identifies efficient subnetworks early in training, reducing computation by 35-45%. These approaches are orthogonal to improved initialization and could potentially be combined with GHN-based methods.

3 Methodology

Our goal is to evaluate whether Graph HyperNetworks can provide effective initialization for Transformer-based language models. Following recent GHN designs, our methodology has three main components: (1) representing Transformer architectures as graphs, (2) using a Graphormer-based GHN to predict parameters, and (3) adapting predicted parameters to the exact shapes required by each layer, using either tiling or interpolation. We then meta-train the GHN on a large distribution of architectures and evaluate on held-out models.

3.1 Graph Representation of Transformers

Each Transformer architecture is represented as a directed computational graph whose nodes correspond to parameterized operations and whose edges capture data dependencies. Nodes include operations such as token and positional embeddings, multi-head

self-attention, linear projections, feed-forward blocks, normalization layers, activation functions, and residual aggregation. Each node is associated with a feature vector encoding its operation type, input and output dimensionality, activation type, and parameter shape.

Edges connect operations that exchange data (for example, a linear projection feeding into an attention block, or a residual branch merging with the main path). We distinguish standard forward edges from residual and positional edges, and we incorporate additional virtual edges based on shortest-path distance to capture long-range interactions. This graph representation allows the GHN to recognize structural patterns shared across different Transformer configurations.

3.2 Graphormer-Based Graph HyperNetwork

To process these graphs, we employ a Graphormer-style architecture. The GHN consists of multiple layers of edge-aware multi-head self-attention interleaved with position-wise feed-forward networks and residual connections. Node features are first embedded and augmented with structural encodings (for example, degree information or distance from input nodes). Attention layers then aggregate information from neighboring nodes and relevant edges, enabling the GHN to build rich representations of the architecture.

After L graph-processing layers, each parameterized node has a final embedding that summarizes its local role and its global position in the architecture. A lightweight decoder then maps node embeddings to parameter tensors. For weight matrices, the decoder produces a canonical two-dimensional tensor, which is subsequently reshaped and adapted to match the required layer dimensions.

3.3 Parameter Adaptation: Tiling vs Interpolation

The GHN predicts parameters at a canonical resolution that does not necessarily match the exact shape of each target layer. We therefore employ two strategies to adapt the predicted tensors to the required shapes, yielding two GHN-based initialization variants.

GHN-T (Tiling). In the tiling-based variant, which we refer to simply as GHN-T in our experiments, predicted weight tensors are expanded to the target size by repeating them along the output and input dimensions. If the target layer is smaller than the canonical resolution, the tensor is cropped; if it is larger, the canonical tensor is tiled until the desired size is reached and then truncated. This approach is computationally cheap and preserves local patterns in the predicted weights but introduces sharp discontinuities at tile boundaries.

GHN-I (Interpolation). In the interpolation-based variant, GHN-I, the predicted tensor is treated as a low-resolution template that is smoothly resized to the target shape. Downscaling is performed using pooling-like averaging, while upscaling uses interpolation (for example, linear or bilinear) to produce continuous transitions across the tensor. This yields smoother and better-conditioned weight matrices that avoid the blocky artifacts of tiling, especially in large layers.

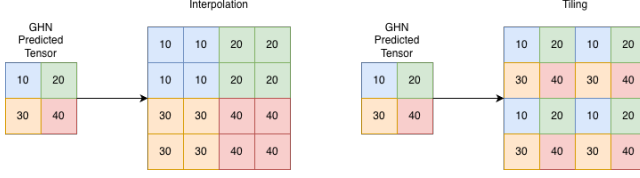


Figure 1: GHN parameter adaptation via interpolation and tiling. Interpolation preserves smooth structure when scaling predicted weights whereas tiling introduces block-wise repetition that can hinder optimization.

Comparison. Tiling emphasizes exact repetition and speed, whereas interpolation emphasizes smoothness and scalability. Our experiments compare both GHN-T (tiling) and GHN-I (interpolation) against a strong standard Transformer initialization.

3.4 Training Dataset and Architecture Sampling

Language data. We use WikiText-2 (wikitext-2-raw-v1) as the underlying language modeling dataset. All text is tokenized using the GPT-2 byte-pair encoding (BPE) tokenizer with a vocabulary size of 50,257 tokens. During GHN meta-training we train on fixed-length subsequences of length T (default $T = 64$), using standard next-token prediction with labels shifted by one position.

Architecture distribution. To train the GHN on a diverse family of Transformer language models, we generate a large programmatic design space of decoder-only “MiniGPT” variants. In total, this design space contains approximately 1.4×10^5 ($\sim 140K$) distinct Transformer architectures before optional filtering, providing a highly diverse meta-training distribution. Each architecture is parameterized by model width d_{model} , depth n_{layer} , number of attention heads n_{head} , and an MLP expansion ratio r (where $d_{\text{ff}} = r \cdot d_{\text{model}}$). We enumerate: (i) $d_{\text{model}} \in \{32, 64, 96, \dots, 2048\}$ (32 values), (ii) $n_{\text{layer}} \in \{1, \dots, 30\}$, (iii) $r \in \{1.0, 1.5, 2.0, \dots, 8.5\}$ (16 values), and (iv) $n_{\text{head}} \in \{1, \dots, 32\}$ subject to the divisibility constraint $d_{\text{model}} \bmod n_{\text{head}} = 0$. Architectures are named as mini-gpt-{L}-{d}-h{h}-r{r} for book-keeping and reproducibility.

3.5 Meta-Training Procedure

The GHN is trained end-to-end to minimize language modeling loss on WikiText-2 when its predicted parameters are used to initialize sampled Transformers. At each meta-training step, we sample a batch of architectures from the configuration space, construct their graphs, predict parameters with the GHN, and compute the next-token cross-entropy loss on text batches. The overall objective combines this task loss with a light regularization term on predicted weights to encourage stability and prevent degenerate solutions.

Loss Function. The GHN is trained via a meta-objective that combines task-specific cross-entropy loss with parameter regularization. For a sampled architecture $\mathcal{M}$ with GHN-predicted parameters $\mathbf{W}_{\mathcal{M}} = \text{GHN}_{\theta}(\mathcal{G}_{\mathcal{M}})$ and a batch of tokens $x_{1:T}$, the meta-training loss is

Table 1: Key GHN meta-training hyperparameters.

Hyperparameter	Value
Optimizer	Adam ($\beta_1 = 0.9, \beta_2 = 0.999$)
Learning rate	4×10^{-4}
Weight decay on predicted weights	$\lambda_{\text{wd}} = 10^{-5}$
Meta-batch size (architectures/step)	$M = 2$
Token batch size (per architecture)	64 sequences
Meta-training epochs	75

Table 2: GHN architecture (hypernetwork) configuration.

Parameter	Value
Graphormer encoder heads	2
Graphormer encoder layers	3
Hidden size (d_{hid})	32
Decoder	conv
Max Tensor Shape	(1024, 1024, 1, 1)

$$\mathcal{L}_{\text{meta}}(\theta) = \mathbb{E}_{\mathcal{M} \sim \mathcal{C}, x_{1:T} \sim \mathcal{D}} \left[- \sum_{t=1}^T \log p_{f_{\theta}(\mathcal{M})}(x_t | x_{<t}) \right] + \lambda_{\text{wd}} \mathbb{E}_{\mathcal{M} \sim \mathcal{C}} [\|\mathbf{W}_{\mathcal{M}}\|_2^2]. \quad (1)$$

where $f_{\theta}(\mathcal{M})$ denotes the instantiated Transformer initialized with GHN-predicted weights, and λ_{wd} is the weight decay applied directly to predicted parameters.

In all GHN meta-training experiments we set the parameter regularization coefficient to $\lambda_{\text{wd}} = 10^{-5}$. We optimize θ with Adam using learning rate 4×10^{-4} , $\beta_1 = 0.9$, $\beta_2 = 0.999$, and train for 75 epochs over the architecture distribution $\mathcal{C}$ and WikiText-2. Each meta-step samples $M = 2$ architectures and uses a batch size of 64 sequences of length 64 tokens per architecture.

Table 2 summarizes the GHN (hypernetwork) architecture used to encode graphs and decode parameter tensors.

Meta-batch. Each meta-training step samples a set of M architectures $\{\mathcal{M}_1, \dots, \mathcal{M}_M\} \sim \mathcal{C}$ and draws one token batch per architecture from WikiText-2. The GHN predicts initialization weights for each $\mathcal{M}_m$, we compute next-token cross-entropy, and backpropagate through the GHN to update θ under the meta-objective in Eq. 1.

3.6 Evaluation Protocol

To assess the effectiveness of GHN-based initialization, we define nine benchmark architectures (B1-B9) that are excluded from the GHN training set to evaluate out-of-distribution generalization. For each configuration, we train three models on WikiText-2:

- **Default:** a baseline using a standard variance-preserving Transformer initialization.
- **GHN-T:** initialization using GHN-predicted parameters with tiling-based adaptation.

Algorithm 1 GHN Meta-Training Procedure

Require: Architecture distribution $\mathcal{C}$, dataset $\mathcal{D}$, GHN parameters θ , weight decay λ_{wd}

```

1: for each training step do
2:   Sample a batch of architectures  $\{\mathcal{M}_1, \dots, \mathcal{M}_M\} \sim \mathcal{C}$ 
3:   Sample token batches  $\{x_{1:T}^{(1)}, \dots, x_{1:T}^{(M)}\} \sim \mathcal{D}$ 
4:   for each architecture  $\mathcal{M}_m$  do
5:     Build its computational graph  $\mathcal{G}_{\mathcal{M}_m}$ 
6:     Predict parameters  $\mathbf{W}_{\mathcal{M}_m} = \text{GHN}_{\theta}(\mathcal{G}_{\mathcal{M}_m})$ 
7:     Instantiate Transformer  $f_{\theta}(\mathcal{M}_m)$  with  $\mathbf{W}_{\mathcal{M}_m}$ 
8:     Compute task loss

$$\mathcal{L}_{\text{task}}^{(m)} = - \sum_{t=1}^T \log p_{f_{\theta}(\mathcal{M}_m)}(x_t^{(m)} | x_{<t}^{(m)})$$

9:   end for
10:  Compute regularization loss  $\mathcal{L}_{\text{reg}} = \lambda_{\text{wd}} \frac{1}{M} \sum_{m=1}^M \|\mathbf{W}_{\mathcal{M}_m}\|_2^2$ 
11:  Total loss:  $\mathcal{L}_{\text{meta}} = \frac{1}{M} \sum_m \mathcal{L}_{\text{task}}^{(m)} + \mathcal{L}_{\text{reg}}$ 
12:  Update GHN parameters:  $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\text{meta}}$ 
13: end for

```

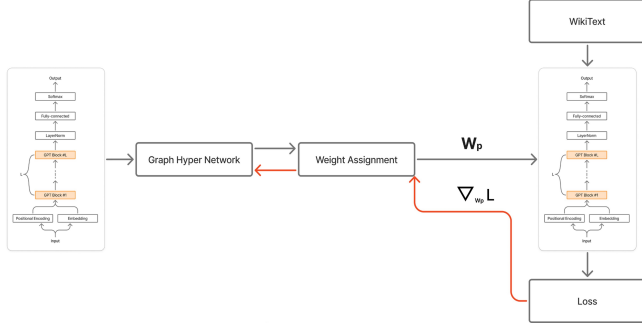


Figure 2: Overview of the GHN training procedure. For each sampled Transformer architecture, we (1) build its computational graph, (2) encode the graph with a Graphormer-based GHN, (3) decode node embeddings into canonical weight tensors, (4) adapt them via interpolation or tiling to match layer shapes, and (5) compute the language modeling loss on WikiText-2 to update GHN parameters.

- **GHN-I:** initialization using GHN-predicted parameters with interpolation-based adaptation.

All models share the same training hyperparameters, optimizer, learning rate schedule, and stopping criterion. For each run, we track validation perplexity over epochs and select the checkpoint with the best validation performance. We report the corresponding test perplexity, the convergence epoch (epoch of best validation perplexity), early-epoch validation perplexity, and a simple generalization gap between validation and test performance. This protocol allows us to evaluate both convergence speed and final quality of each initialization method.

4 Experimental Results

We evaluate GHN-based initialization on nine held-out Transformer configurations ranging from 5M to 100M parameters. For each configuration, we compare three initialization schemes: (1) a standard variance-preserving Transformer initialization (**Default**), (2) GHN with tiling-based parameter adaptation (**GHN-T**), and (3) GHN with interpolation-based adaptation (**GHN-I**). All models are trained on WikiText-2 using identical optimization settings.

Perplexity (PPL) is reported as the exponentiated average negative log-likelihood:

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t})\right), \quad (2)$$

where θ denotes the model parameters and T is the sequence length. Lower perplexity indicates a better language model.

4.1 Train Perplexity and Convergence Speed

Table 3 reports the train perplexity at convergence and the convergence epoch for all nine benchmarks. Default initialization converges the fastest across nearly all configurations, typically within 5-8 epochs. GHN-I is consistently slower, converging in 7-13 epochs, while GHN-T requires the most epochs, with convergence ranging from roughly 17-27 epochs for Tiny/Small models and remaining above 11 epochs even for Large models.

The train perplexity at the convergence epoch shows that GHN predictions produce reasonably learnable parameterizations: across all benchmarks, GHN-I and GHN-T achieve train perplexities between 78-123, compared to 52-102 for Default. Although GHN-based schemes start from substantially worse effective initial losses, they still converge to competitive ranges, confirming that GHN-predicted weights are functional initializations.

Table 4 further analyzes three representative configurations (B3, B6, B9) under different amounts of SGD fine-tuning. Without any fine-tuning, GHN-I substantially improves over Random across all three model sizes, and the gap persists even after 3 and 10 epochs of SGD. GHN-T, in contrast, performs poorly in all regimes. These results indicate that GHN-predicted parameters are immediately more informative than random weights, but the quality of the adaptation strategy (interpolation vs. tiling) has a large impact on downstream perplexity.

4.2 Test Perplexity

Test perplexity results mirror the trends observed in convergence behavior. Across all nine benchmarks, Default achieves the strongest final perplexities, followed by GHN-I, with GHN performing worst. The relative degradation in test perplexity increases with model size: while GHN-I exhibits a modest 10-15% degradation on smaller models, it degrades by 20-30% on the largest configurations.

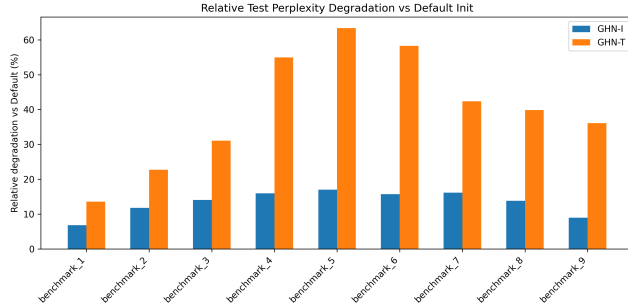
Figure 3 visualizes this trend, showing that both GHN-I and GHN-T incur larger relative degradation as the parameter count increases.

These results suggest that interpolation-based parameter adaptation (GHN-I) provides noticeably smoother and more stable initializations compared to tiling (GHN), particularly as hidden dimensions grow.

Table 3: Train perplexity at convergence and convergence epoch across Benchmark 1-9 for Default, GHN-I, and GHN-T initialization. Lower values are better.

Benchmark	Train perplexity at convergence ↓			Convergence epoch ↓		
	Default	GHN-I	GHN-T	Default	GHN-I	GHN-T
Benchmark 1	102.13	115.84	112.43	20	19	27
Benchmark 2	80.38	96.12	92.43	12	13	19
Benchmark 3	67.26	97.84	90.82	10	10	17
Benchmark 4	65.97	78.29	98.61	8	10	15
Benchmark 5	67.57	78.78	123.11	6	8	11
Benchmark 6	67.21	80.46	101.41	6	8	13
Benchmark 7	58.37	78.54	82.71	6	7	14
Benchmark 8	57.92	78.40	89.79	6	7	13
Benchmark 9	52.76	82.53	88.33	6	6	12

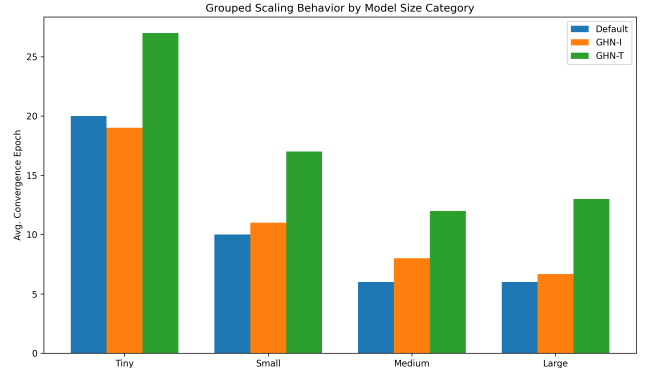
	No Fine-Tuning			3 Epoch of SGD			10 Epoch of SGD		
	B3 (17M)	B6 (43M)	B9 (100M)	B3 (17M)	B6 (43M)	B9 (100M)	B3 (17M)	B6 (43M)	B9 (100M)
Random	957.6	635.6	584.5	506.1	339.4	313.2	169.6	130.8	113.0
GHN-I	702.1	550.6	476.0	423.7	327.8	274.8	161.5	141.6	110.0
GHN-T	1152.0	818.0	700.5	560.8	430.0	389.5	234.8	224.9	211.3

Table 4: Perplexity comparison across initialization methods under different amounts of SGD fine-tuning. Lower is better.**Figure 3: Relative degradation in test perplexity for GHN-I and GHN-T compared to Default initialization across all nine benchmarks. Lower values indicate closer performance to the baseline.**

4.3 Scaling Trends

To study how initialization quality scales with model size, we group the nine benchmarks into four size categories: Tiny (< 10M parameters, B1-B2), Small (10-30M, B3-B4), Medium (30-60M, B5-B6), and Large (> 60M, B7-B9). Figure 4 reports the average convergence epoch for each category and initialization method.

Default initialization exhibits a clear scaling trend: convergence accelerates as model size increases, dropping from roughly 20 epochs for Tiny models to about 6 epochs for Medium and Large models. GHN-I follows a similar pattern but remains consistently 1-2 epochs slower than Default in every category. GHN-T is the

**Figure 4: Average convergence epoch for Default, GHN-I, and GHN-T initialization when grouping benchmarks into Tiny (< 10M), Small (10-30M), Medium (30-60M), and Large (> 60M) models. Default converges fastest in all regimes; GHN-I is consistently slower but improves with scale, while GHN-T lags behind across all categories.**

slowest across the board, converging very slowly on Tiny models and only partially catching up for larger architectures.

This grouped view highlights that our GHN learns qualitatively useful weight structures that benefit larger models, but it also makes clear that meta-learned initialization does not yet match a strong Transformer baseline, especially when tiling is used for parameter adaptation.

5 Analysis

Our results highlight several insights about the behavior and limitations of Graph HyperNetworks when applied to Transformer-based language models.

5.1 Effectiveness of GHN Predictions

GHN-predicted weights produce models that train successfully across all benchmarks, demonstrating that computational-graph-based parameter prediction is feasible for Transformers. However, the performance gap relative to Default initialization shows that GHNs trained on small architectures do not yet capture the full complexity of weight distributions required for larger attention layers and feed-forward blocks.

5.2 Tiling vs. Interpolation

A key observation is the strong performance difference between GHN (tiling) and GHN-I (interpolation):

- **GHN-I consistently outperforms GHN-T** on both convergence speed and final perplexity.
- Tiling introduces abrupt block discontinuities in large matrices, causing optimization to spend additional steps smoothing out artifacts.
- Interpolation preserves global structure while scaling smoothly, yielding better-conditioned initializations.

These findings align with intuition: Transformers rely heavily on finely-tuned attention projections, and discontinuous tiled weights hinder early gradient flow.

5.3 Architectural Sensitivity

GHN generalization appears particularly sensitive to hidden dimension and model depth. Although the GHN was meta-trained across diverse small architectures (128-512 dimensions), its predictions degrade for large 768-dimensional models. This suggests that the canonical parameter resolution used during GHN training may be insufficient to capture the high-rank structure of large attention maps.

5.4 Comparison to Prior Vision-Based GHNs

Prior work such as GHN-3 demonstrated strong zero-shot accuracy improvements for CNNs on ImageNet, even for deep architectures never encountered during training. In contrast, our results show noticeably weaker transfer in the Transformer setting. We attribute this to:

- the more intricate interaction patterns in attention layers,
- the sensitivity of Transformer training to weight scale and alignment,
- the mismatch between small meta-training architectures and large-scale evaluation architectures.

This highlights the need for Transformer-specific GHN design or larger-scale meta-training to bridge the gap.

6 Future Work

Our findings suggest that improving GHN initialization for language models will require scaling both the meta-training distribution and the hypernetwork itself.

Scaling the meta-training architecture distribution. While our design space contains $\sim 140\text{K}$ MiniGPT variants (before optional filtering), prior GHN work scales meta-training to substantially larger pools of architectures (on the order of 10^6 models) and benefits from broader architectural diversity [7]. Expanding the Transformer meta-training space to include more challenging regimes (wider/deeper models, richer attention patterns, and larger feed-forward expansions) may help the GHN learn weight structure that transfers more reliably to large language-model settings.

Training larger GHNs for longer with more compute. A second direction is to increase GHN capacity (e.g., more graph-encoder layers, larger hidden size, and higher-resolution canonical tensors) and train for longer meta-horizons. For Transformers, learning stable attention and MLP initialization patterns may require both greater model capacity and longer meta-training than in our current setup. Exploring multi-resolution decoding and Transformer-specific regularization/objectives is also a promising way to improve scaling behavior.

In summary, scaling toward millions of architectures [7], increasing GHN capacity, and extending meta-training duration are concrete paths toward closing the performance gap and learning stronger architecture-aware initializations for Transformer language models.

7 Conclusion

This work presents the first systematic investigation of Graph HyperNetworks (GHNs) for initializing Transformer-based language models. We introduce a graph-based representation of Transformer architectures, apply a Graphormer-style GHN to predict initialization weights, and study two parameter adaptation strategies—tiling (GHN-T) and interpolation (GHN-I)—across nine held-out architectures on WikiText-2.

Empirically, GHN-predicted weights are *functional*: all evaluated models train stably, and GHN-I consistently improves over random initialization in zero-shot and early-epoch settings. However, GHN-based initialization does not yet match a strong variance-preserving Transformer baseline. Relative to Default, GHN-I typically converges 1-2 epochs later and yields higher final perplexity, while GHN-T underperforms most strongly. The gap widens with model scale, suggesting that accurate attention/MLP weight structure is difficult to meta-learn under limited GHN capacity and a restricted meta-training regime.

Overall, our results highlight both the promise and current limitations of architecture-aware, meta-learned initialization for Transformers, and they point to clear scaling directions needed to close the gap to strong hand-designed initialization baselines.

References

- [1] Tianlong Chen, Jonathan Frankle, Shiyu Chang, Sijia Liu, Yang Zhang, Zhangyang Wang, and Michael Carbin. 2021. The lottery ticket hypothesis for pre-trained bert networks. *Advances in neural information processing systems* 33 (2021), 15834–15846.
- [2] Yann N Dauphin and Samuel S Schoenholz. 2019. Metainit: Initializing learning by learning to initialize. *Advances in Neural Information Processing Systems* 32 (2019).
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

- [4] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 249–256.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*. 1026–1034.
- [6] Boris Knyazev, Dong-Ki Hwang, and Simon Lacoste-Julien. 2021. Parameter prediction for unseen deep architectures. *Advances in Neural Information Processing Systems* 34 (2021), 5857–5869.
- [7] Boris Knyazev, Dong-Ki Hwang, and Simon Lacoste-Julien. 2023. Can We Scale Transformers to Predict Parameters of Diverse ImageNet Models?. In *International Conference on Machine Learning*. PMLR, 17366–17397.
- [8] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843* (2017).
- [9] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [11] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do transformers really perform badly for graph representation? *Advances in Neural Information Processing Systems* 34 (2021), 28877–28888.
- [12] Yang You, Jing Li, Sashank Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, Kurt Keutzer, and Cho-Jui Hsieh. 2019. Large batch optimization for deep learning: Training bert in 76 minutes. *arXiv preprint arXiv:1904.00962* (2019).
- [13] Chris Zhang, Mengye Ren, and Raquel Urtasun. 2018. Graph hypernetworks for neural architecture search. In *International Conference on Learning Representations*.
- [14] Hongyi Zhang, Yann N Dauphin, and Tengyu Ma. 2019. Fixup initialization: Residual learning without normalization. *arXiv preprint arXiv:1901.09321* (2019).
- [15] Hao Zhang, Yijun Hao, Xiaoge Zhang, Xin Dong, Shanshan Wang, and Ning Ding. 2020. Progressively stacking 2.0: A multi-stage layerwise training method for bert training speedup. *arXiv preprint arXiv:2011.13635* (2020).
- [16] Chen Zhu, Renkun Xu, Michael Pfister, and Zheng Chen. [n. d.]. GradInit: Learning to initialize neural networks for stable and efficient training.